

Triangle Counting by Adaptively Resampling over Evolving Graph Streams

Wei Xuan, Huawei Cao, Mingyu Yan, Zhimin Tang,
Xiaochun Ye and Dongrui Fan

Institute of Computing Technology, Chinese Academy of Sciences, China

SEKE 2021
July 1 - July 10, 2021

Outline

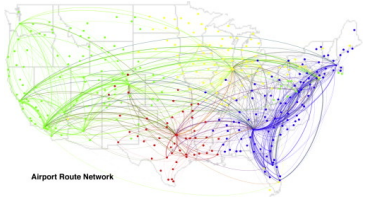
- **Introduction**
- Related Work
- Proposed Method
- Experiments
- Conclusions

Triangle Counting in a Graph

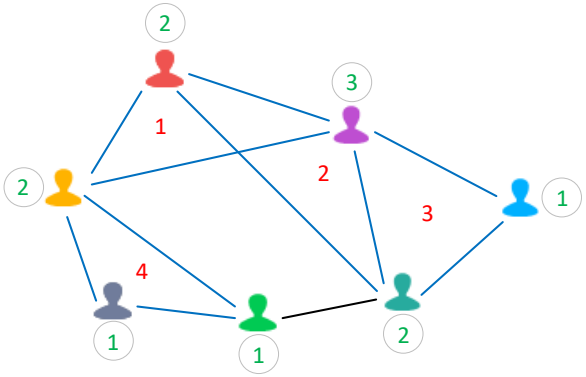
- Graphs are ubiquitous
  - vertex: entity, edge: relation between entities
  - social networks、E-commerce networks、traffic networks







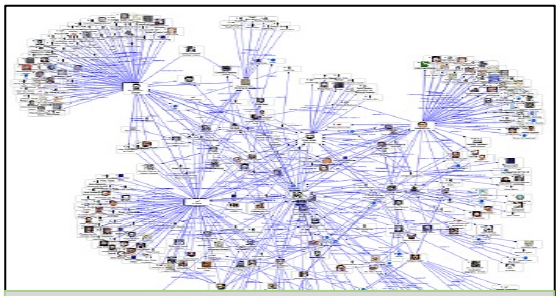
- Triangle counting
  - Global triangles: 3-node connected to each other
  - Local triangles: triangles for each vertex



Applications of Triangle Counting



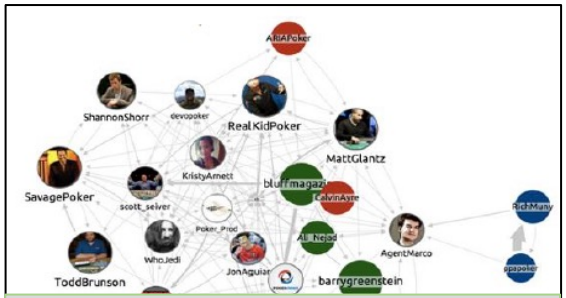
Financial Fraud Detection



Community Discovery



Recommended System



Social Network Analysis



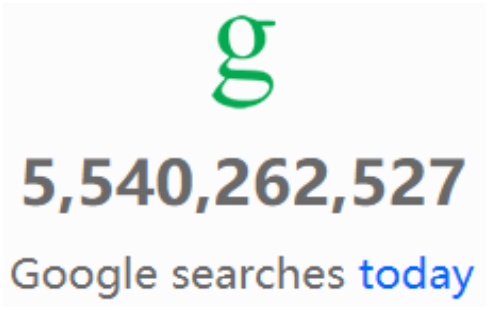
Traffic Network

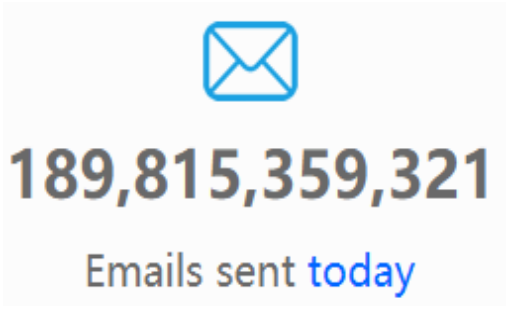


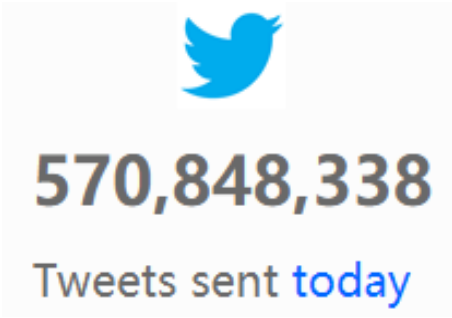
Spam Detection

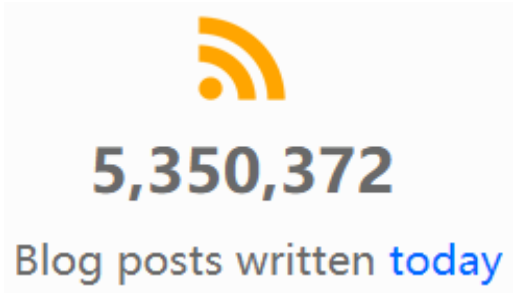
Evolving Graph Streams

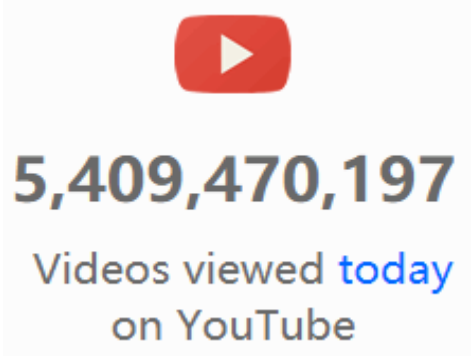
- Real-world graphs are dynamic
  - Stream of edges
  - Adversarial order

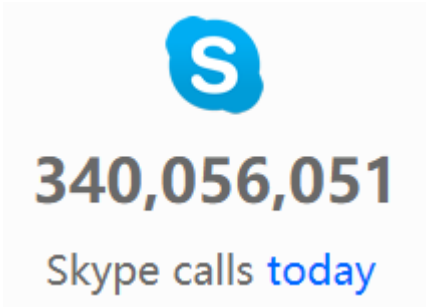






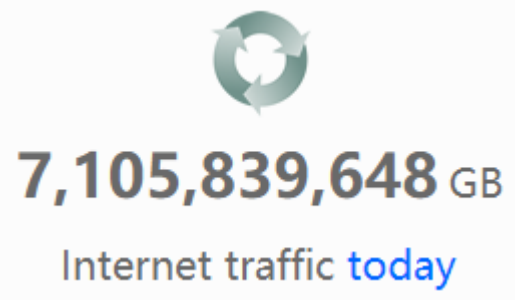


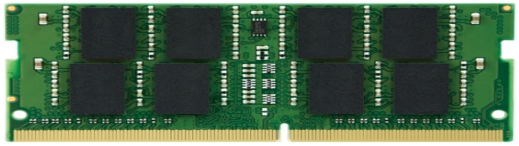




Challenges in Streaming Graphs

- Massive real-world graphs
- Growing upwards as a whole
- Limited memory space





Outline

- Introduction
- **Related Work**
- Proposed Method
- Experiments
- Conclusions

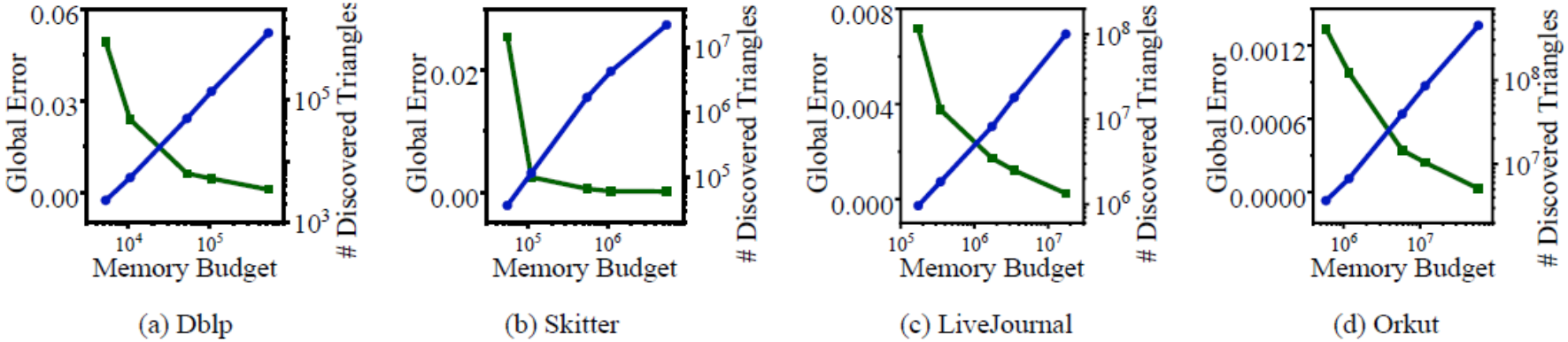
Related Work

- Two categories based on memory allocation strategy
  - Reservoir-based method
  - Bernoulli-based method

Sampling Method	Sampling Ratio	Memory Budget
BSR-Sample (Proposed)	**≥Threshold**	**Adaptive**
Reservoir-based sample	Decreased	Fixed
Bernoulli-based sample	Fixed	Increased

Reservoir-based method

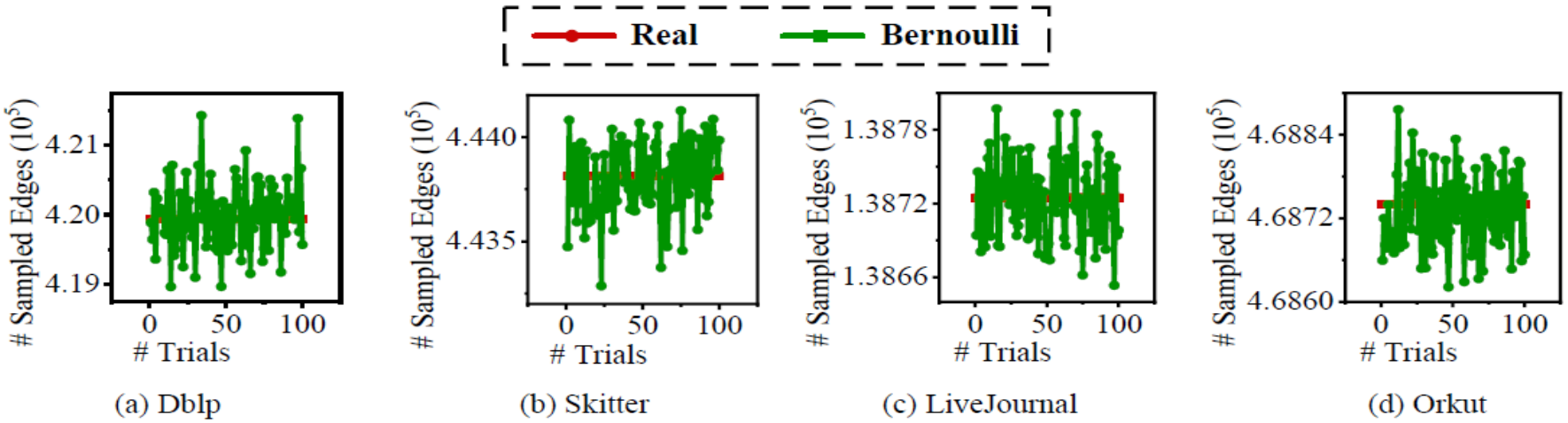
- State-of-the-art approaches
  - TRIEST: first handle fully-dynamic graph streams
  - WRS: employ temporal locality
  - ThinkD: improve TRIEST by “Counting first then Sampling”
- Accuracy
  - affected by memory budget size



As the memory memory increases, global error decreases and the number of discovered triangles increases.

Bernoulli-based method

- State-of-the-art approaches
  - gSH: a sampling framework using different probabilities
  - GPS: a sampling framework by setting different priority
  - MASCOT: local triangle estimation
- Consumed memory
  - fluctuated around the real value
  - affected by initial sampling probability



Outline

- Introduction
- Related Work
- **Proposed Method**
- Experiments
- Conclusions

Observations

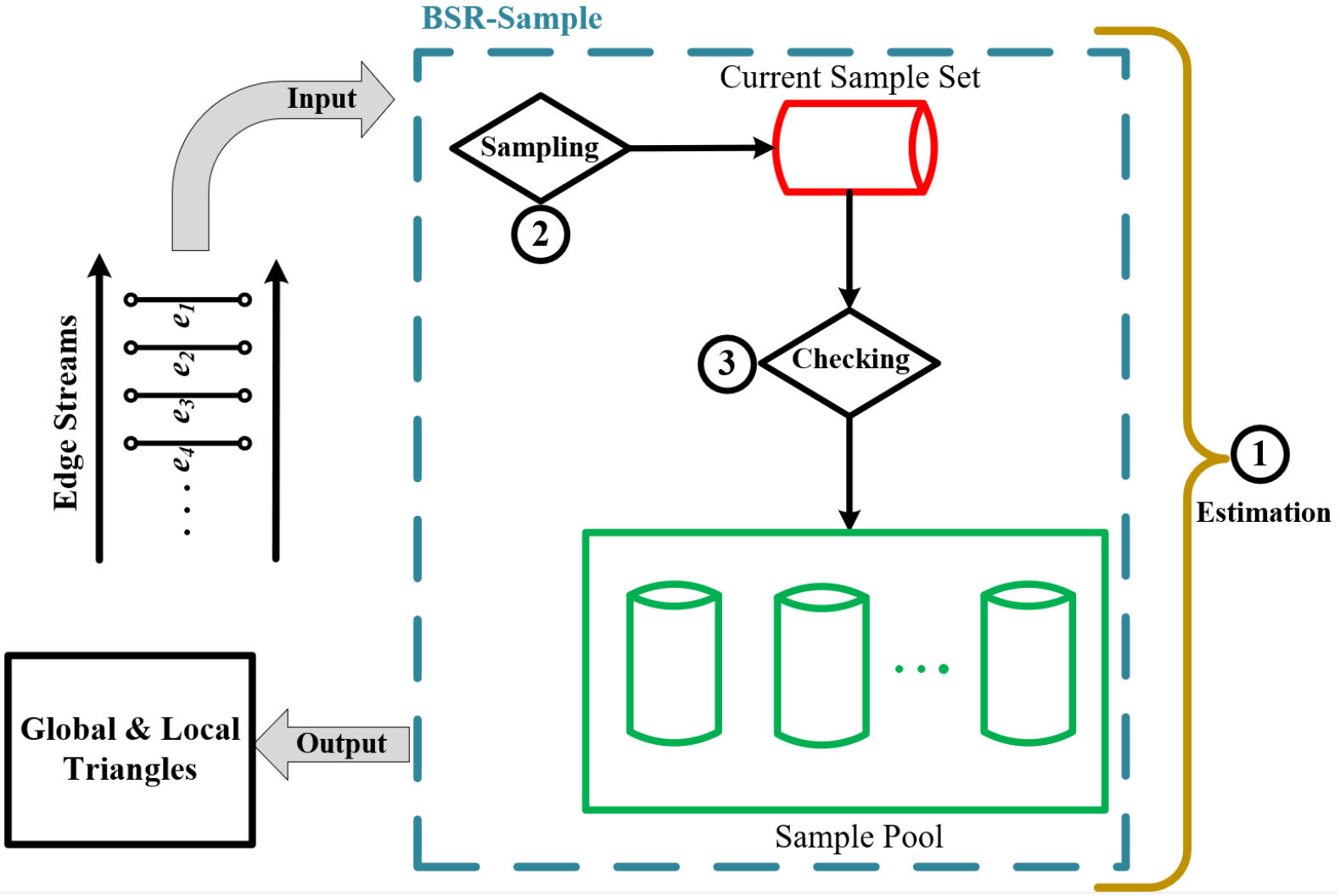
- Scale of graph streams is unknown in advance
- Evolving graph streams usually grow upwards
- Limited main memory space

✓ How to maintain an appropriate **sampling ratio** and how much **memory space** to allocate for accurate global and local triangle counting?

Goals

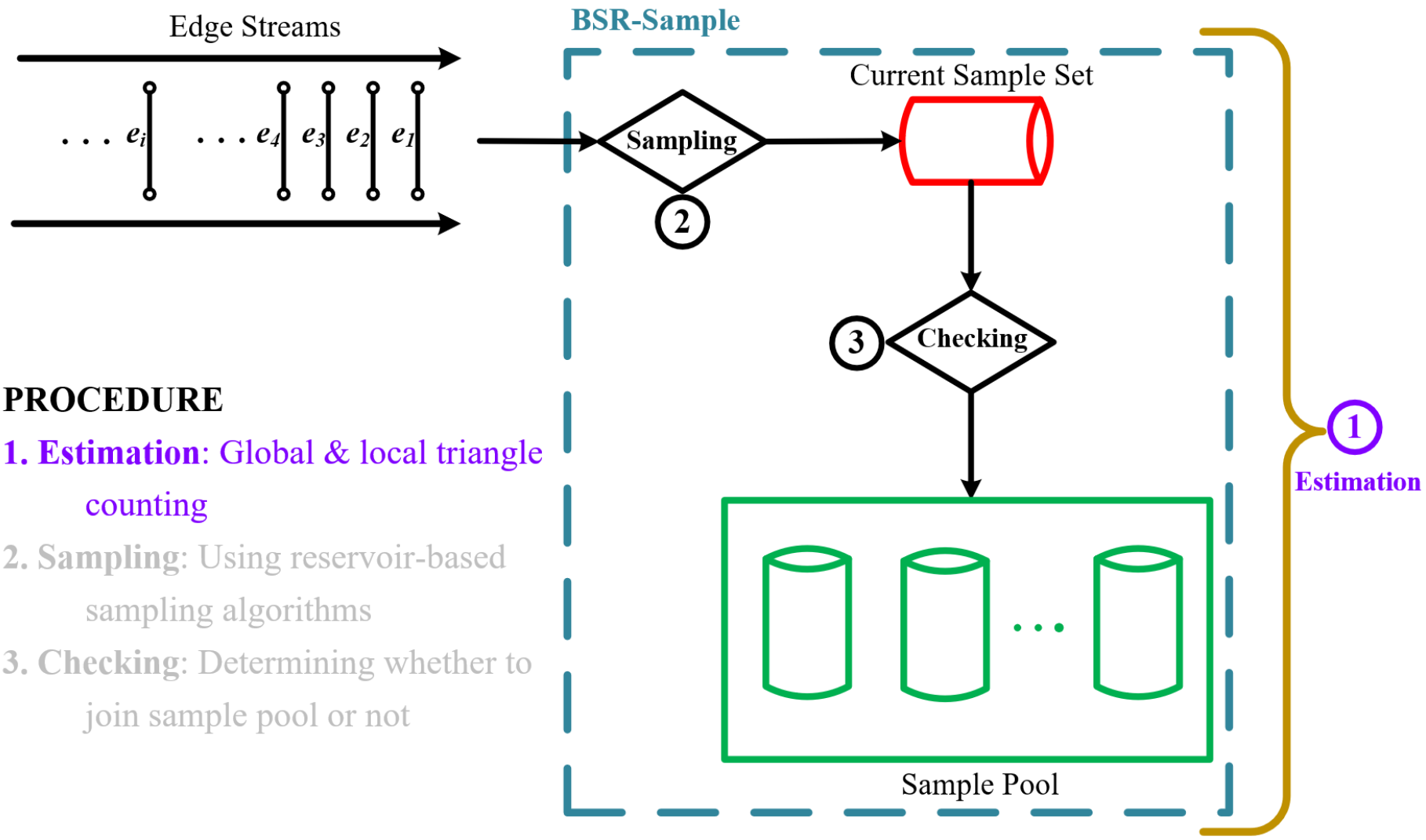
- Maintain a bounded-sampling-ratio
  - achieve accurate and stable estimation of global and local triangle counting
- Control Consumed memory budget
  - improve the efficiency of memory usage
- Develop single-pass streaming algorithm
  - avoid multi-repeated and redundant operations
- Process only in memory space
  - no need to access slower secondary storage

Overview



The framework of BSR-Sample & BSR-TC

Estimation

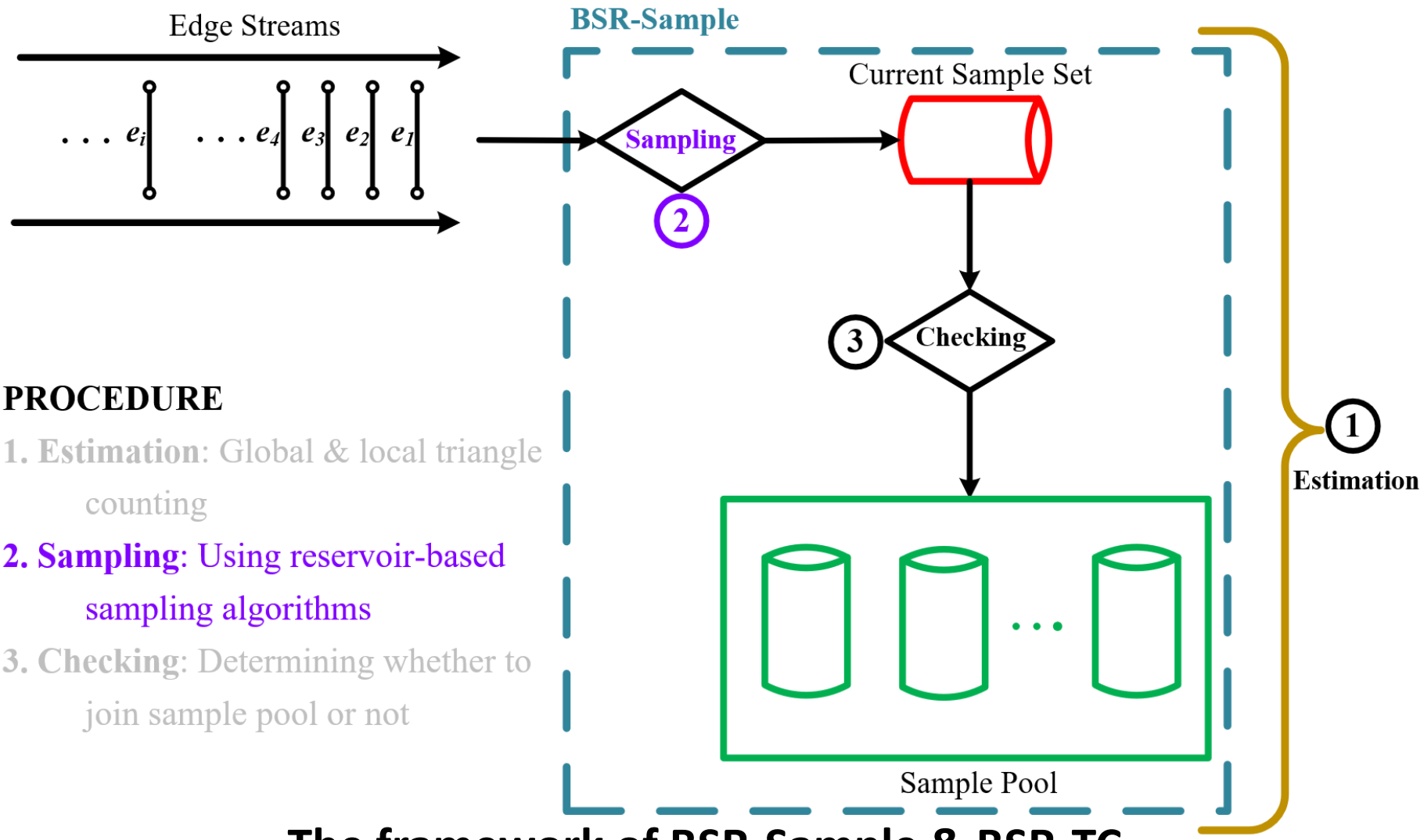


PROCEDURE

1. **Estimation**: Global & local triangle counting
2. **Sampling**: Using reservoir-based sampling algorithms
3. **Checking**: Determining whether to join sample pool or not

The framework of BSR-Sample & BSR-TC

Sampling

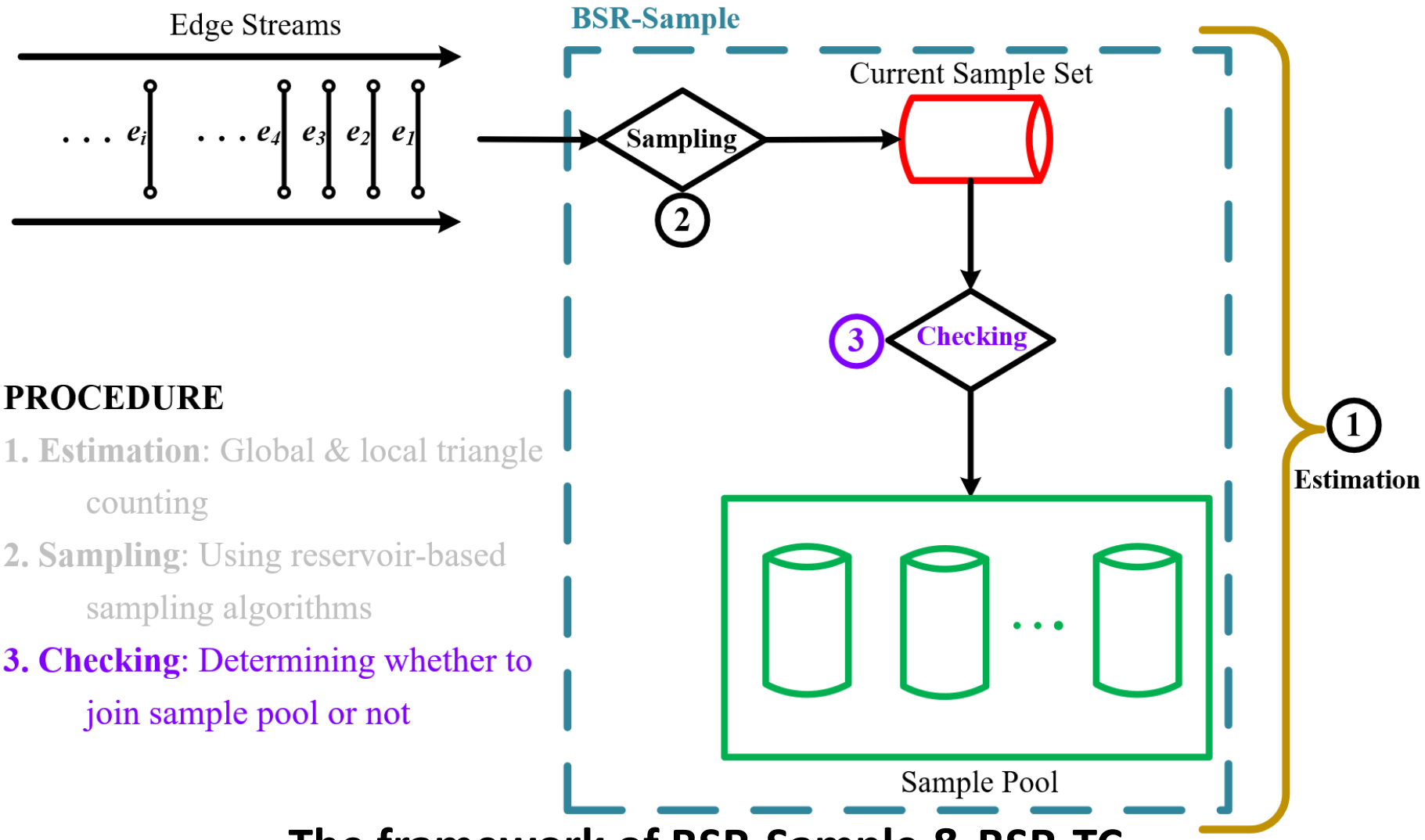


PROCEDURE

1. **Estimation**: Global & local triangle counting
2. **Sampling**: Using reservoir-based sampling algorithms
3. **Checking**: Determining whether to join sample pool or not

The framework of BSR-Sample & BSR-TC

Checking



PROCEDURE

1. **Estimation**: Global & local triangle counting
2. **Sampling**: Using reservoir-based sampling algorithms
3. **Checking**: Determining whether to join sample pool or not

The framework of BSR-Sample & BSR-TC

Algorithms

Algorithm 2: BSR-Sample for Triangle Counting (BSR-TC)

Input: $\mathcal{S}$: set of sampled edges.
Output: (1) Δ: global triangle counting;
(2) Δ_u: local triangle counting for node u.

$\Delta \leftarrow 0$
for *each new arriving edge* $e_t = (u, v)$ **do**
  CountTriangle$((u, v))$.
  SampleEdge$((u, v), \mathcal{S}_c)$.
  CheckRatio$((u, v), \mathcal{S}_c, \mathcal{S}_p)$.
end
Function `CountTriangle` $((u, v))$
  if $u \notin \mathcal{V}$ **then**
    $\mathcal{V} \leftarrow \mathcal{V} \cup \{u\}$ and $\Delta_u \leftarrow 0$
  end
  if $v \notin \mathcal{V}$ **then**
    $\mathcal{V} \leftarrow \mathcal{V} \cup \{v\}$ and $\Delta_v \leftarrow 0$
  end
  $\mathcal{N}^{\mathcal{S}}_{u,v} \leftarrow \mathcal{N}^{\mathcal{S}}_{u} \cap \mathcal{N}^{\mathcal{S}}_{v}$
  for *each* $c \in \mathcal{N}^{\mathcal{S}}_{u,v}$ **do**
    $\Delta \leftarrow \Delta + 1/p_{cuv}$; $\Delta_c \leftarrow \Delta_c + 1/p_{cuv}$;
    $\Delta_u \leftarrow \Delta_u + 1/p_{cuv}$; $\Delta_v \leftarrow \Delta_v + 1/p_{cuv}$;
  end
End

Algorithm 1: Bounded-Sampling-Ratio Sample (BSR-Sample)

Input: (1) $\{e^{(1)}, e^{(2)}, \cdots\}$: a graph stream;
(2) $\mathcal{M}$: initial memory budget;
(3) $\mathcal{R}$: specified sampling ratio threshold.
Output: $\mathcal{S}$: a set of sampled edges.

for *each new arriving edge* $e_t = (u, v)$ **do**
  SampleEdge$((u, v), \mathcal{S}_c)$.
  CheckRatio$((u, v), \mathcal{S}_c, \mathcal{S}_p)$.
end
Function `SampleEdge` $(e_t, \mathcal{S}_c)$
  $t_c \leftarrow t \,\%\, \mathcal{M}$
  if $t_c \leq \mathcal{M}$ **then**
    $\mathcal{S}_c \leftarrow \mathcal{S}_c + \{(u, v)\}$
  end
  else if *a generated random number* $(0, 1) \leq \mathcal{M}/t_c$ **then**
    choose a random edge (m, n) from $\mathcal{S}_c$
    $\mathcal{S}_c \leftarrow \mathcal{S}_c - \{(m, n)\}$
    $\mathcal{S}_c \leftarrow \mathcal{S}_c + \{(u, v)\}$
  end
End
Function `CheckRatio` $(e_t, \mathcal{S}_c, \mathcal{S}_p)$
  $t_c \leftarrow t \,\%\, \mathcal{M}$
  if $\mathcal{M}/t_c = \mathcal{R}$ *and* $\Theta \geq \mathcal{M}$ **then**
    remove the current sampling set $\mathcal{S}_c$ into sampling pool $\mathcal{S}_p$
    create a new $\mathcal{S}_c$ with size $\mathcal{M}$
    $\Theta \leftarrow \Theta - \mathcal{M}$
  end
End

Outline

- Introduction
- Related Work
- Proposed Method
- **Experiments**
- Conclusions

Evaluation

Name	# Nodes	# Edges	Summary
Dblp	317,080	1,049,866	Collaboration network
Skitter	1,696,415	11,095,298	Internet topology graph
LiveJournal	3,997,962	34,681,189	Friendship network
Orkut	3,072,441	117,185,083	Online social network

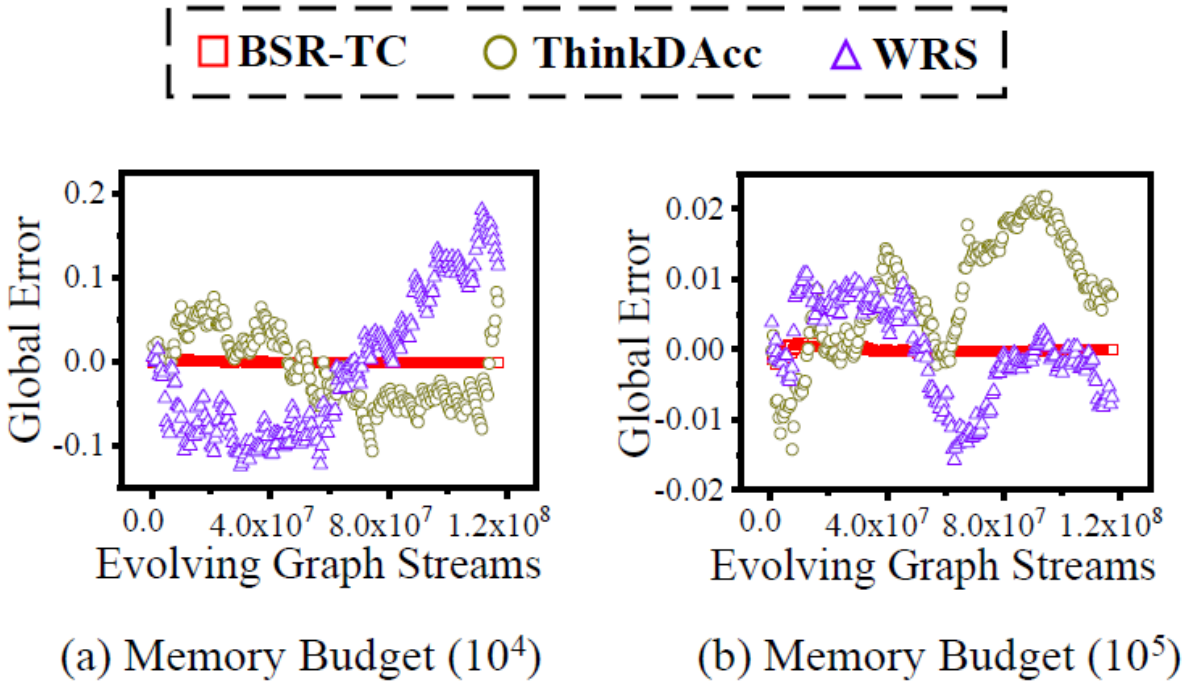
- Platforms
  - A server with Intel Xeon Gold 6148 processors
  - 64-bit Red Hat Linux OS
- Datasets
  - 4 real-world graph datasets
- Baseline
  - Three state-of-the-art triangle counting algorithms: WRS, MASCOT and ThinkDAcc
- Metrics
  - Global error
  - Local Error

$$\frac{1}{c}\sum_{i=1}^{c}\frac{|\hat{x}-x|}{\hat{x}+1}$$

$$\frac{1}{c}\sum_{i=1}^{c}\left\{\frac{1}{|\mathcal{V}|}\sum_{u\in\mathcal{V}}\frac{|\hat{x}_u-x_u|}{\hat{x}_u+1}\right\}$$

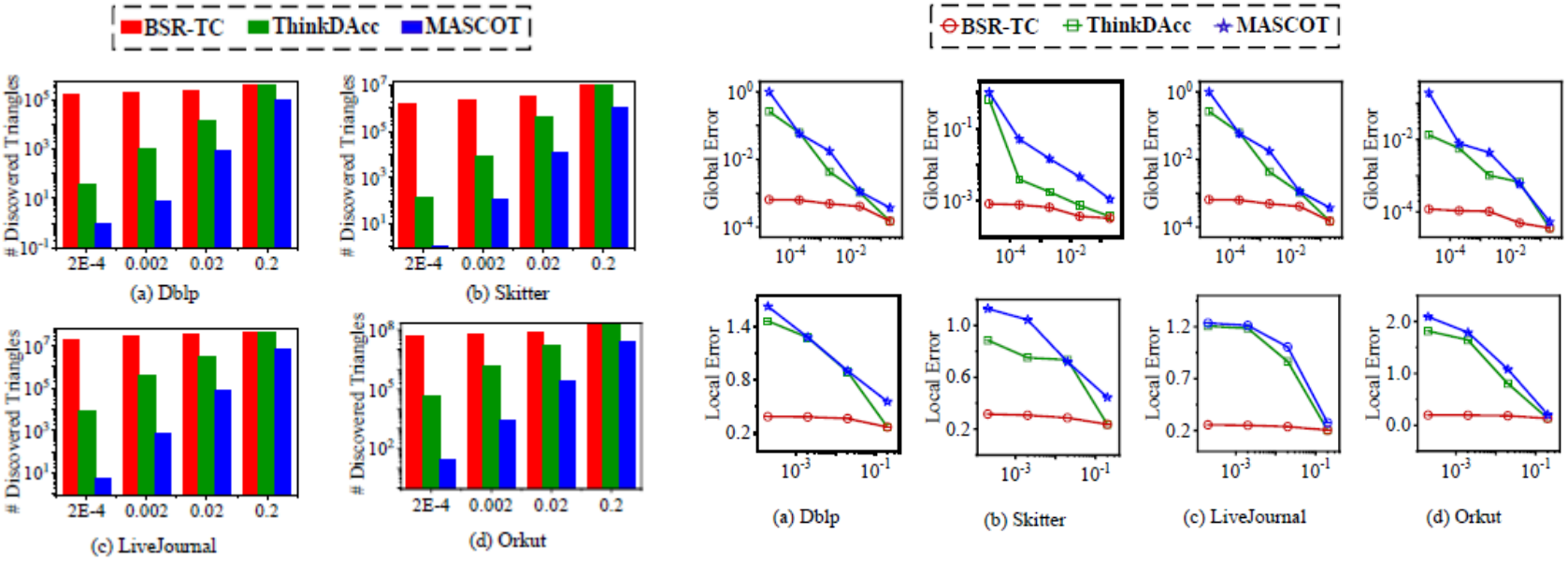
Scalability

- Maintain accuracy over evolving graph streams
  - BSR-TC has unbiased estimates
  - BSR-TC keeps stable



Accuracy

- Estimation Errors
  - BSR-TC discovers orders of magnitude more triangles
  - BSR-TC reduces both global and local errors



Outline

- Introduction
- Related Work
- Proposed Method
- Experiments
- **Conclusions**

Conclusions

- Method: novel and general sampling
  - Adaptively resampling
  - Sampling ratio is always ≥ user-specified one
  - without any prior knowledge about graph streams
- Algorithms: global and local triangle counting
  - BSR-Sample(Bounded-Sampling-Ratio Sample)
  - BSR-TC(Bounded-Sampling-Ratio for triangle counting)
- Analyses: scalability and accuracy
  - BSR-TC has accurate and unbiased results

Thanks!